
Designing and Deploying Systematic Trading Strategies for Prediction Markets

Gareth Flynn
Independent Research Project

Abstract

This report summarises an independent research project on Polymarket's short-duration Bitcoin Up/Down contracts. I reconstructed the contracts' external reference price from multi-venue cryptocurrency data, tested statistical and machine-learning strategies, and deployed selected approaches in a live event-driven trading system. Live testing showed that execution latency could remove an apparent backtested edge, shifting the research focus from predictive accuracy towards latency-aware expected value.

1 Motivation and Research Process

Prediction markets provide a compact setting in which probability modelling, market microstructure and execution interact directly. I initially set out to understand whether Polymarket's Bitcoin contracts could be modelled systematically, with the longer-term aim of extending the framework to other contract horizons and underlying assets.

Rather than begin with a preferred model, I followed an end-to-end research process:

market mechanics → data → strategy research → backtesting → live execution → iteration.

The central lesson was that a signal is only useful if it survives data limitations, transaction costs and executable latency.

2 Resolution Modelling and Data

The contracts resolve against a Chainlink reference price using an undisclosed aggregation of at least three different cryptocurrency exchanges, rather than a print from one exchange. I therefore built a resolution-window collector covering 17 BTC quote pairs across 12 exchanges. It sampled top-of-book data every 25 ms, stored top-10 book snapshots when venues changed, and normalised USD, USDT and USDC quotes onto a common dollar basis.

The proxy was evaluated against completed Polymarket outcomes. On the 15-minute dataset, 227 boundaries produced 206 matched contracts and 412 endpoint observations. The best five-venue combination achieved an MAE of \$2.34 and RMSE of \$3.04. On a later five-minute sample of 282 resolutions, a five-venue proxy achieved an MAE of \$1.53 and RMSE of \$2.31.

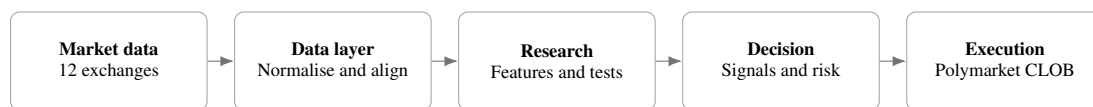


Figure 1: Research and live-execution pipeline.

3 Strategy Research

I compared rule-based strategies—including scalping, market making and statistical arbitrage—with Q-learning and LSTM-based approaches. Strategies were assessed on signal stability, downside exposure, transaction costs and whether the decision could be executed under live market conditions. Only the more viable, lower-risk candidates were progressed to deployment.

The system detected directional moves over rolling windows, applied contract-time, price and adverse-move filters, and required a fresh threshold move after a blocked or consumed signal to avoid trading stale events.

4 Live System and Execution Research

I implemented the selected strategies as an asynchronous, event-driven Python system. Polymarket books were maintained from its market WebSocket after a REST snapshot, with reconnection and snapshot fallbacks. Orders were submitted as fill-and-kill limits through the CLOB.

The implementation included:

- cached signer and token-risk state, persistent HTTP sessions and warmed connections;
- separate worker pools for general CLOB calls, entries and urgent exits;
- controls for duplicate orders, in-flight entries, stops and failed-sell retries;
- delayed on-chain balance reconciliation to detect fills after a timeout; and
- structured logging of book age, signing and post times, signal-to-fill latency, slippage, fees and net P&L.

Analysis of 2,394 historical order attempts quantified the main execution constraint. The fill rate was 50.7%, median HTTP post duration was 1,799 ms, and the median upper bound on signal-to-fill time for filled orders was 1,825 ms. Filled orders executed a median \$0.02 above the signal-time ask. For failed orders, the median maximum ask increase while the request was pending was \$0.08, compared with \$0.03 for filled orders.

Table 1: Selected measured results from the research and live-system logs.

Metric	Result
Resolution collector	17 pairs across 12 exchanges
15-minute proxy validation	412 endpoints; MAE \$2.34
5-minute proxy validation	282 resolutions; MAE \$1.53
Historical order attempts	2,394
Observed fill rate	50.7%
Median HTTP post duration	1,799 ms
Median filled signal-to-fill upper bound	1,825 ms

These observations changed the trading objective. A signal should be evaluated against the likely price after execution delay, not only the price visible when the signal is generated:

$$\mathbb{E}[\Pi_t] = \mathbb{E}[V_t - P_{t+\tau}] - C_{\text{fees}} - C_{\text{slippage}} > 0,$$

where V_t is estimated contract value and $P_{t+\tau}$ is the executable price after random delay τ .

5 Ongoing Research

Current work focuses on strategies that retain positive expected value under observed execution conditions:

- **Latency-aware backtesting:** sampling empirical order delays and replaying the intervening order-book path rather than assuming immediate fills.
- **Signal-decay modelling:** estimating how quickly the edge following an external exchange move disappears and rejecting signals whose expected half-life is shorter than the execution path.

- **Execution improvements:** testing pre-signed orders, persistent connections, geographically closer hosting (initial tests using a VPS have shown positive results) and maker execution where fill probability justifies the lower cost.
- **Robust strategy selection:** comparing entry filters, stop rules, position caps and drawdown behaviour after fees using replay and counterfactual analysis.
- **Broader markets:** extending the resolution proxy and strategy framework across five- and 15-minute contracts, longer-duration markets and other underlying assets as reliable data becomes available.

The project remains under active development. Its principal finding is that predictive modelling, data construction and execution cannot be assessed independently: the relevant target is executable expected value under realistic market conditions.